Shiv Nadar University - Chennai

Department of Computer Science

25CS2741 Fundamentals of Internet of Things + Lab

Date: 10/08/2026 (Week 4) **Continuous Lab Evaluation-5** II Yr/ 3rd Sem B. Tech CSE(IoT)A&B

Allotted Marks: 10 (Hardware Implementation + Code Efficiency + Viva-Voce)

Topics: 8051 Tool Chain setup – Development Board – AT89S52

Strictly No to using **Arduino APIs, HAL libraries, RTOS services!**

Supporting Documents: 1. Board Schematics 2. Pin diagram of 8051

Hardware Debug: Multi meter.

Hardware Timer and Interrupt-Based Timing for LED Fading, Pulse Generation, and Stepper Motor Control

Problem Statement:

- Design a timer-driven embedded control system using the 8051 in which a hardware timer periodically generates interrupts.
- Use the timer interrupt as the system time base to control LED brightness, generate a variable-frequency pulse signal, and control the stepping sequence of a stepper motor.
- The implementation must be non-blocking and must not use software delay loops for timing.

Task:

(1) LED fading using software PWM

(2) variable-frequency square-wave generation,

(3) Stepper-motor speed and direction control. (*)

(*) Optional for this Week.

Details of Task1:

Configure Timer 0 to generate a precise periodic interrupt. Use the timer interrupt to create a software time base and toggle an LED every 1 second. Verify the generated timing using an oscilloscope.

Crystal (11.0592 MHz)
↓
Machine Cycle
↓
Timer Counter
↓
Overflow
↓
Interrupt
↓
Software Time Base
↓
1-second LED control

For a classic 12-clock 8051:

$$T_{machine} = \frac{12}{f_{crystal}}$$

For example, with an 11.0592 MHz crystal:

$$T_{machine} \approx 1.085\ \mu s$$

For a 1 ms interrupt:

$$N = \frac{1ms}{1.085\mu s} \approx 922$$

Timer preload:

$$65536 - 922 = 64614 = FC66_H$$

Therefore:

```c
TH0 = 0xFC;
TL0 = 0x66;
```

Hardware generates the timing; software responds to the timing event.

Details of Task2:

Using the timer interrupt developed in Task1, generate a software PWM signal and gradually vary the duty cycle from 0% to 100% and back to 0%, producing a visible LED fading effect.

Eg: 0% 10% 20% 30% 40% 50% 60% 70% 80% 90% 100% 90% 80% ... 10% 0%

Suggested PWM frequency

Use approximately: **500 Hz – 1 kHz**

The LED should appear continuously illuminated rather than visibly flickering.

Key concept

For example:

Duty Cycle	LED Brightness
0%	OFF
25%	Dim
50%	Medium
75%	Bright
100%	Fully ON

Configure a GPIO pin as a signal output and use the timer interrupt to generate square-wave signals of different frequencies. Measure the output frequency and duty cycle using an oscilloscope.

Mode 1 → 100 Hz; Mode 2 → 500 Hz; Mode 3 → 1 kHz; Mode 4 → 2 kHz; Mode 5 → 5 kHz

Different Signal Types

Once the square wave works, ask students to generate several simple waveforms.

1. Square Wave 101010101010 **2. Variable Duty-Cycle Wave** 1111000011110000	**3. Pulse Train** 100000001000000010000000 **4. Burst Signal** 1010101010 0000000000 1010101010

Details of Task3: Stepper Motor

Timer interrupt ↓ Generate next step ↓ 1000 ↓ 0100 ↓ 0010 ↓ 0001	The **timer interval determines motor speed**. For example: **Slow:** 100 ms/step **Medium:** 50 ms/step **Fast:** 10 ms/step